

Documentação Técnica — Modulo3_Orquestrador_Autonomo

Arquitetura, componentes, configuração e exemplos de uso

21 de maio de 2026

Documentação Técnica — Modulo3_Orquestrador_Autor

1. Visão Geral

O Modulo3_Orquestrador_Autonomo é um orquestrador de requisições que integra os módulos de dispatcher, router, adapters, handlers e persistência em um pipeline coeso e extensível. O projeto foi criado em 17 de maio de 2026, contém 33 arquivos distribuídos em 10 diretórios, totalizando 26.450 bytes. Sua principal característica é operar com zero dependências externas — toda a implementação utiliza apenas a biblioteca padrão do Python (stdlib puro).

O orquestrador recebe uma requisição padronizada contendo identificador, tipo e dados; submete-a a validação e normalização; roteia o request para o adapter adequado; executa o handler com fallback opcional; registra auditoria e decisões em disco; e retorna uma resposta estruturada. O design modular permite registrar novos adapters sem modificar o núcleo, e a configuração externa em JSON possibilita ajustes sem reinicialização.

2. Arquitetura do Sistema

A arquitetura segue o padrão de pipeline sequencial com pontos de decisão. Cada requisição percorre as etapas de validação, normalização, roteamento, dispatch, execução, fallback (se necessário) e logging antes da resposta ser retornada ao solicitante.

2.1 Diagrama de Fluxo

Request !' ValidateRequest !' NormalizeRequest !' RoutingEngine

!“

RequestDispatcher !' HandlerRegistry (seleciona adapter)

!“

!“

FallbackStrategy !' CoreAdapter (se necessário)

!“

AuditLogger + DecisionTracker

!“

Response

O fluxo começa com a validação dos campos obrigatórios (id, type, data). Após normalização (inserção de context e normalização de chaves para lowercase), o **RoutingEngine** determina qual adapter deve processar o request com base no campo type. O **RequestDispatcher** consulta o **HandlerRegistry** para obter a classe do adapter, executa o método handle() e captura o resultado. Se o handler primário falhar, o **FallbackStrategy** redireciona para o **CoreAdapter**. Ao final, o **AuditLogger** registra request e response, e o **DecisionTracker** armazena a decisão de roteamento.

2.2 Componentes Principais

Componente	Arquivo	Responsabilidade
HandlerRegistry	handlers/handler_registry.py	Gerencia o mapeamento type !' handler; registra adapters por tipo (cli, rpa, core, expert, skill); permite lookup via get_handler(type).
RequestDispatcher	dispatcher/request_dispatcher.py	Recebe request e handler type; executa handler.handle(request); retorna response.
RoutingEngine	router/routing_engine.py	Interpreta request['type'] para determinar adapter alvo; usa o tipo do request como chave de roteamento.
OrquestradorCore	orchestrator/orchestrator_core.py	Integra todos os componentes; método process(request) executa pipeline completo.

FallbackStrategy	orchestrator/fallback_strategy.py	Define fallback para core_adapter quando handler primário falha; executa CoreAdapter.handle(request) com flag de fallback.
ExecutionContext	orchestrator/execution_context.py	Objeto de contexto compartilhado; request_id gerado automaticamente; tracking de adapter selecionado, fallback usado, timestamps.

2.3 HandlerRegistry

O **HandlerRegistry** é o repositório central de handlers disponíveis. Ele mantém um dicionário interno que associa uma string identificadora (ex.: "cli") à classe do adapter correspondente. Através do método `register(type, handler_class)` novos adapters podem ser adicionados em tempo de execução. O método `get_handler(type)` retorna a classe registrada ou levanta uma exceção se o tipo não existir. Este registro é utilizado pelo `RequestDispatcher` para instanciar e executar o handler correto.

2.4 RequestDispatcher

O **RequestDispatcher** encapsula a lógica de execução. Ele recebe o request original e o identificador do handler (obtido do `RoutingEngine`). Internamente, chama `handler_class.handle(request)` — que é um método estático definido em todos os adapters — e retorna o dicionário de resposta produzido pelo adapter. O dispatcher não realiza validação ou transformação dos dados; sua única função é conectar o roteamento à execução.

2.5 RoutingEngine

O **RoutingEngine** implementa a lógica de decisão de rota. Ele analisa o campo `type` da requisição e o utiliza como chave para selecionar o adapter alvo. O mecanismo é compatível com as regras definidas no arquivo **orchestrator_config.json**, que lista os handlers habilitados e seus prefixes. Se o tipo solicitado não corresponder a nenhum handler registrado, o engine pode delegar ao adapter padrão definido na configuração.

2.6 OrquestradorCore

O **OrquestradorCore** é o componente central que integra todos os demais. Seu método `process(request)` executa o pipeline completo:

1. Cria um **ExecutionContext** com `request_id` único e timestamp de início.
2. Utiliza o **RoutingEngine** para determinar o tipo de handler.
3. Aciona o **RequestDispatcher** para executar o handler selecionado.
4. Em caso de falha, aciona o **FallbackStrategy** que redireciona para o **CoreAdapter**.
5. Registra o log de auditoria via **AuditLogger**.
6. Registra a decisão de roteamento via **DecisionTracker**.
7. Retorna um dicionário contendo a resposta do adapter, o `execution_context` e flags de sucesso/fallback.

2.7 FallbackStrategy

O **FallbackStrategy** fornece resiliência ao sistema. Quando o handler primário indicado pelo roteamento falha (por exemplo, adapter não registrado, erro interno ou timeout), a estratégia de fallback redireciona a requisição para o **CoreAdapter**. O **CoreAdapter** executa o processamento com uma flag `fallback: true` no contexto, permitindo que o sistema recupere a requisição sem interromper o fluxo. A configuração de fallback pode ser ajustada no arquivo `fallback_rules.json` (planejado para versões futuras).

2.8 ExecutionContext

O **ExecutionContext** é um objeto de contexto compartilhado que acompanha toda a vida da requisição. Ele contém:

Atributo	Tipo	Descrição
<code>request_id</code>	<code>str</code>	Identificador único gerado automaticamente via <code>uuid4</code> .
<code>handler_type</code>	<code>str</code>	Tipo do adapter selecionado (ex.: <code>"cli"</code> , <code>"core"</code>).
<code>fallback_used</code>	<code>bool</code>	Indica se o fallback foi acionado.

start_time	float	Timestamp de início do processamento.
end_time	float	Timestamp de fim do processamento.

O contexto é populado progressivamente ao longo do pipeline e retornado ao solicitante junto com a resposta, permitindo rastreabilidade completa.

3. Adaptadores

3.1 BaseAdapter

A classe base abstrata que define o contrato para todos os adapters. Possui um método estático `handle(request)` que recebe um dicionário e retorna um dicionário. Todos os adapters concretos herdam de **BaseAdapter** e implementam sua própria lógica de `handle`.

3.2 CLIAdapter

O **CLIAdapter** executa comandos do sistema operacional. O campo `data.command` contém o comando shell a ser executado. O método `handle` utiliza `subprocess.run()` para executar o comando, captura a saída padrão e retorna um dicionário com o resultado.

Exemplo: `request {"data": {"command": "ls -la"}}` ! resposta com a listagem

3.3 RPAAdapter

O **RPAAdapter** simula ações de automação robótica de processos. O campo `data.task` especifica a tarefa a ser simulada (ex.: "extract", "click", "type"). O retorno é um dicionário com a descrição da ação simulada e um indicador de sucesso.

Exemplo: `request {"data": {"task": "extract"}}` ! resposta "RPA task executed"

3.4 CoreAdapter

O **CoreAdapter** realiza operações matemáticas e de sistema. O campo `data.operation` define a operação (ex.: "sum", "multiply"). Para a operação "sum", os campos `data.a` e `data.b` são somados e o resultado retornado. É o adapter utilizado como fallback padrão.

```
Exemplo: request {"data": {"operation": "sum", "a": 2, "b": 3}} !' resposta
```

3.5 ExpertAdapter

O **ExpertAdapter** realiza análises especializadas com base em uma consulta textual. O campo `data.query` contém a pergunta ou termo a ser analisado. O retorno é uma string de análise pré-formatada.

```
Exemplo: request {"data": {"query": "performance"}} !' resposta "Expert a
```

3.6 SkillAdapter

O **SkillAdapter** executa uma skill específica registrada no sistema. O campo `data.skill_name` identifica qual skill deve ser executada. O retorno é um dicionário com o resultado daquela skill.

```
Exemplo: request {"data": {"skill_name": "test_skill"}} !' resposta "Skill e
```

4. Persistência

4.1 AuditLogger

O **AuditLogger** registra todas as requisições e respostas em formato JSONL (JSON Lines). O método `log_request(request)` persiste o dicionário de requisição, e `log_response(response)` persiste o dicionário de resposta. O diretório de log é configurável; por padrão, utiliza `logs/audit.jsonl`. O diretório é criado automaticamente se não existir.

4.2 DecisionTracker

O **DecisionTracker** rastreia as decisões de roteamento tomadas pelo orquestrador. O método `log_decision(request, handler_type)` registra o identificador da requisição e o tipo de handler selecionado, permitindo auditoria posterior das decisões de roteamento. O arquivo padrão é `logs/decisions.jsonl`.

5. Utilitários

5.1 Validators

A função `validate_request(request)` verifica a presença dos campos obrigatórios: `id`, `type` e `data`. Retorna `True` se todos os campos estiverem presentes, `False` caso contrário. Utilizada no início do pipeline para rejeitar requisições malformadas.

5.2 Normalizers

A função `normalize_request(request)` garante que o campo `context` exista (inserindo um dicionário vazio se ausente) e converte todas as chaves do dicionário `data` para `lowercase`, padronizando a entrada independente do formato original. Retorna o `request` modificado.

6. Configuração

6.1 orchestrator_config.json

O arquivo de configuração central do orquestrador define parâmetros de execução e a lista de handlers habilitados.

Parâmetro	Valor	Descrição
<code>max_retries</code>	3	Número máximo de tentativas para handlers com falha.
<code>timeout_seconds</code>	30	Tempo máximo de execução de um handler.
<code>default_adapter</code>	"core"	Adapter padrão quando o tipo não é reconhecido.
<code>logging.level</code>	"INFO"	Nível de logging do sistema.
<code>logging.directory</code>	"logs"	Diretório para arquivos de log.
<code>handlers.cli.enabled</code>	true	Habilita o CLIAdapter.
<code>handlers.rpa.enabled</code>	true	Habilita o RPAAdapter.
<code>handlers.core.enabled</code>	true	Habilita o CoreAdapter.
<code>handlers.expert.enabled</code>	true	Habilita o ExpertAdapter.

8. Instalação e Setup

Para instalar e executar o orquestrador, siga os passos abaixo:

1. **Clone ou extraia** projeto para o diretório desejado.
2. **Instale as dependências**— o arquivo requirements.txt está vazio, pois o projeto utiliza apenas a biblioteca padrão. Nenhum pacote externo é necessário.
3. **Execute os testes** com o comando pytest a partir da raiz do projeto. Quatro suites de teste estão disponíveis: test_adapters.py — 5 testes unitários para cada adapter.
4. test_persistence.py — 2 testes para AuditLogger e DecisionTracker.
5. test_utils.py — 2 testes para validators e normalizers.
6. test_integration.py — 1 teste de integração completo.
7. **Configure** o arquivo config/orchestrator_config.json conforme necessário antes da execução.

9. Exemplo de Uso

O código a seguir demonstra a criação do orquestrador, registro de adapters e processamento de uma requisição do tipo CLI:

```
from orchestrator.orchestrator_core import OrchestratorCore

from dispatcher.request_dispatcher import RequestDispatcher

from router.routing_engine import RoutingEngine

from orchestrator.fallback_strategy import FallbackStrategy

from orchestrator.execution_context import ExecutionContext

from handlers.handler_registry import HandlerRegistry

from adapters.cli_adapter import CLIAdapter

from adapters.rpa_adapter import RPAAdapter

from adapters.core_adapter import CoreAdapter

from adapters.expert_adapter import ExpertAdapter

from adapters.skill_adapter import SkillAdapter
```

Registro dos adapters

```
registry = HandlerRegistry()

registry.register("cli", CLIAdapter)

registry.register("rpa", RPAAdapter)

registry.register("core", CoreAdapter)

registry.register("expert", ExpertAdapter)

registry.register("skill", SkillAdapter)
```

Criação dos componentes

```
dispatcher = RequestDispatcher(registry)

router = RoutingEngine()

fallback = FallbackStrategy()

context = ExecutionContext()
```

Orquestrador integrado

```
orchestrator = OrchestratorCore(dispatcher, router, fallback, context)
```

Requisição de exemplo

```
request = {

  "id": "exemplo-001",<br/>

  "type": "cli",<br/>

  "data": {"command": "echo 'Hello, World!'"}

}

response = orchestrator.process(request)

print(response)
```

Saída esperada:


```
{"success": true, "adapter": "cli", "result": "Hello,  
World!"  
", ...}
```

Após a execução, o **AuditLogger** terá registrado a requisição e a resposta no arquivo logs/audit.jsonl, e o **DecisionTracker** terá registrado a decisão de roteamento em logs/decisions.jsonl.

10. Manutenção

O sistema foi projetado para ser facilmente extensível e mantido. As seguintes diretrizes aplicam-se:

- **Extensão:** Para adicionar um novo adapter, crie uma classe que herde de `BaseAdapter` e implemente o método estático `handle(request)`. Registre a nova classe no `HandlerRegistry` e inclua a configuração no `orchestrator_config.json` se desejado.
- **Configuração:** As configurações em JSON (como `orchestrator_config.json`) são lidas em tempo de inicialização. Para aplicar alterações sem reiniciar o processo, é necessário recarregar o arquivo — funcionalidade que pode ser adicionada conforme necessidade.
- **Debugging:** Os logs em formato JSONL são legíveis por qualquer ferramenta de análise textual. O campo `request_id` permite correlacionar requisições, decisões e respostas.
- **Encoding:** Todos os arquivos de configuração e código devem ser mantidos em **UTF-8**, garantindo compatibilidade com caracteres acentuados e símbolos especiais.